

Spec — Guest Access Requests: Notification + Daily Briefing

mini-moi · Guild Created: 2026-06-14 — Claude.ai Status: spec_ready For: Claude Code
Depends on: spec_security_architecture_2026-06-14.md (testable end-to-end once app.minimoi.ai is live; build + local test in parallel)

What this is

When someone submits a guest access request, Robert gets a Telegram ping and the request appears on the Guild Daily Briefing with a status of **requested**, **granted**, or **rejected**. That's the full scope for now.

Part 1 — Database

New table: `guest_requests`

```
sql

CREATE TABLE guest_requests (
  id          SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  email       TEXT NOT NULL UNIQUE,
  reason      TEXT,
  requested_at TIMESTAMPTZ DEFAULT NOW(),
  status      TEXT DEFAULT 'requested', -- requested | granted | rejected
  actioned_at TIMESTAMPTZ,
  last_nudged_at TIMESTAMPTZ          -- set by Loop F after each nudge
);
```

Part 2 — Flask: `/register` route update

Form fields (update registration template):

```
html
```

```
<input type="text" name="name" placeholder="Your name" required>
<input type="email" name="email" placeholder="Your email" required>
<textarea name="reason" placeholder="What brings you here? (optional)"
```

Route logic:

python

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form.get('name')
        email = request.form.get('email')
        reason = request.form.get('reason', '')

        # Log to DB
        req_id = db.insert_guest_request(name, email, reason)

        # Telegram ping
        send_telegram(
            f"Guest access request\n"
            f"Name: {name}\n"
            f"Email: {email}\n"
            f"Reason: {reason or 'not provided'}"
        )

        return render_template('register_thanks.html')
    return render_template('register.html')
```

```
register_thanks.html: "Thanks, [name]. Robert will be in touch."
```

Part 3 — Status update route

A minimal owner-only route to toggle status. No separate admin UI page — just a POST action callable from the Daily Briefing or directly:

python

```
@app.route('/admin/guest-requests/<int:req_id>/status', methods=['POST'])
@owner_required
def update_guest_request_status(req_id):
    status = request.form.get('status') # 'granted' or 'rejected'
    if status in ('granted', 'rejected'):
        db.update_guest_request_status(req_id, status)
    return redirect(url_for('guild_briefing'))
```

The Daily Briefing (see Part 4) renders the action buttons inline — no separate page needed.

Part 4 — Guild Daily Briefing: Access Requests section

New section on `/guild`, between Career Focus and Build. Hidden when no requests exist.

ACCESS REQUESTS

Sarah Chen	2h ago	requested	[Grant] [Reject]
Mark Torres	3d ago	granted	
Alex Kumar	5d ago	rejected	

Display rules:

- **requested** rows: show [Grant] and [Reject] buttons (POST to status route above)
- **granted / rejected** rows: status label only, no action buttons
- Show all requests (not just pending) so Robert has a full log in view
- Email not shown (name only — briefing may be visible on screen during demos)
- Sort: requested first (by requested_at desc), then actioned

CoS function:

```
python
```

```
def get_guest_requests():  
    return db.query("""  
        SELECT id, name, reason, requested_at, status, actioned_at  
        FROM guest_requests  
        ORDER BY  
            CASE status WHEN 'requested' THEN 0 ELSE 1 END,  
            requested_at DESC  
        """)
```

Section hidden entirely when table is empty.

Part 5 — CoS Loop F: Staleness Nudge

Guest requests are time-sensitive during active job search (contract ends August 3, 2026 — any request in the next 8 weeks could be from an interviewer or recruiter). CoS should surface stale requests before they drift, not after.

Rule

If any `guest_requests` row has `status = 'requested'` and `requested_at` is more than **24 hours ago**, CoS fires a Telegram nudge — once per request per day.

python

```

def check_stale_guest_requests():
    stale = db.query("""
        SELECT id, name, requested_at,
               EXTRACT(EPOCH FROM (NOW() - requested_at))/3600 AS hours_pending
        FROM guest_requests
        WHERE status = 'requested'
              AND requested_at < NOW() - INTERVAL '24 hours'
              AND (last_nudged_at IS NULL
                   OR last_nudged_at < NOW() - INTERVAL '23 hours')
        ORDER BY requested_at ASC
    """)

    if not stale:
        return

    if len(stale) == 1:
        r = stale[0]
        msg = (
            f"🕒 Guest request pending {int(r['hours_pending'])}h\n"
            f"{r['name']} is still waiting.\n"
            f"https://app.minimoi.ai/guild (Access Requests section)"
        )
    else:
        lines = [f"🕒 {len(stale)} guest requests pending:"]
        for r in stale:
            lines.append(f"    · {r['name']} - {int(r['hours_pending'])}h")
        lines.append("https://app.minimoi.ai/guild")
        msg = "\n".join(lines)

    send_telegram(msg)

    # Update last_nudged_at for all nudged rows
    ids = [r['id'] for r in stale]
    db.query("UPDATE guest_requests SET last_nudged_at = NOW() WHERE id = ANY(%s)")

```

Scheduler: hourly interval inside CoS service

CoS should be monitoring continuously — not just at 07:30. Guest requests in particular are time-sensitive: a recruiter who submits a request at 2 PM and hears nothing for 24h because the check only runs at dawn is a missed opportunity during active job search.

Approach: internal scheduler inside the CoS Flask service (port 8769)

Use Python's `threading.Timer` (or `APScheduler` if already a dependency) to run `check_stale_guest_requests()` on a **60-minute interval** inside the CoS service process. No new launchd job needed — the CoS service is already running continuously.

```
python

# cos_service.py - add to startup
import threading

def run_hourly_checks():
    """Runs every 60 minutes inside the CoS service."""
    check_stale_guest_requests()
    # Future hourly checks added here (career pipeline, etc.)
    threading.Timer(3600, run_hourly_checks).start()

# Call once at startup after app init
run_hourly_checks()
```

Note on Phase 4 loop wiring: the full loop wiring (Loops B/C/D) is deferred, but this hourly check is intentionally simpler — it lives inside the existing CoS service process, not a separate orchestrated loop. When Phase 4 wiring ships, `run_hourly_checks()` becomes the internal scheduler that feeds into the loop architecture. No rework needed.

Staleness threshold: nudge fires when a request has been pending > **2 hours** with no action (not 24h — during active job search, 2h is the right threshold for a first nudge). Subsequent nudges: once every **6 hours** if still unactioned (`(last_nudged_at < NOW() - INTERVAL '6 hours')`).

Update the query accordingly:

```
sql

WHERE status = 'requested'
  AND requested_at < NOW() - INTERVAL '2 hours'
  AND (last_nudged_at IS NULL
       OR last_nudged_at < NOW() - INTERVAL '6 hours')
```

-
- ☐ `guest_requests` table created
 - ☐ `/register` logs to DB and fires Telegram ping
 - ☐ `register_thanks.html` created
 - ☐ `/admin/guest-requests/<id>/status` POST route (owner-only)
 - ☐ Guild Daily Briefing renders Access Requests section

- ☐ Pending requests show [Grant] / [Reject] buttons
- ☐ Actioned requests show status label only
- ☐ Section hidden when no requests exist
- ☐ Email not shown in briefing

CoS staleness nudge:

- ☐ `last_nudged_at` column added to `guest_requests`
- ☐ `run_hourly_checks()` scheduler added to CoS service startup
- ☐ `check_stale_guest_requests()` runs every 60 minutes inside CoS
- ☐ First nudge fires after **2h** pending with no action
- ☐ Subsequent nudges fire every **6h** if still unactioned
- ☐ Multiple stale requests → one combined message
- ☐ `last_nudged_at` updated after each nudge

End-to-end test:

- ☐ Submit a test request via `/register`
 - ☐ Telegram ping received
 - ☐ Request appears on `/guild` as "requested"
 - ☐ Robert clicks [Grant] → status updates to "granted", buttons disappear
 - ☐ Robert clicks [Reject] on a second test → status updates to "rejected"
-

Commit

```
bash
```

```
git add portal/ templates/ migrations/  
git commit -m "feat: guest access requests – notification + Daily Briefing + CoS  
staleness nudge"  
guest_requests table (requested/granted/rejected, last_nudged_at). /register  
logs request and fires Telegram ping. Guild Daily Briefing: Access Requests  
section with inline Grant/Reject actions for pending requests, status labels  
for actioned ones. Section hidden when empty. Loop F extended: 24h staleness  
nudge via Telegram, once per request per day, combined message for multiple  
stale requests."  
git push origin main
```

Deferred (ROADMAP)

- Auto-match requester email domain to career pipeline
- Full /admin/guest-requests list + detail page
- Guest account auto-creation on Grant (currently manual)

Spec · CoS Guest Access · 2026-06-14